THE YLANG BOOK

A detailed beginner-friendly guide to YLang 2.0.0

Stable core language, step-by-step examples, standard library, troubleshooting, and experimental extensions

VERSION 2.0.0 STABLE

Standalone book edition - detailed chapters maintained with the source repository

The default compiler generates C and uses GCC or Clang. Experimental extensions are labelled separately and do not share the stable-core compatibility guarantee.

Contents

Chapter 1. What is YLang? . . . 3
Chapter 2. Installing YLang . . . 4
Chapter 3. Your First Program . . . 6
Chapter 4. The Compiler Commands . . . 7
Chapter 5. Variables and Constants . . . 8
Chapter 6. Types . . . 10
Chapter 7. Printing and F-Strings . . . 11
Chapter 8. Operators and Expressions . . . 12
Chapter 9. Making Decisions with if . . . 14
Chapter 10. Repeating Things: loop, while, and for . . . 15
Chapter 11. Functions . . . 17
Chapter 12. Arrays . . . 18
Chapter 13. Converting Between Types . . . 19
Chapter 14. The Math Library . . . 20
Chapter 15. Strings, Paths, and Input/Output . . . 21
Chapter 16. Reading Compiler Errors and Using fix . . . 23
Chapter 17. Runtime Errors . . . 24
Chapter 18. A Complete Program . . . 25
Chapter 19. Multi-File Imports (Experimental) . . . 26
Chapter 20. Structs and Enums (Experimental) . . . 27
Chapter 21. GUI and Game2D (Experimental) . . . 28
Chapter 22. The NASM Backend (Experimental) . . . 30
Chapter 23. Platform Support and YSPM Packages . . . 31
Chapter 24. What YLang 2.0.0 Does Not Have . . . 32
Chapter 25. Quick Reference . . . 33
Chapter 26. Practice and Next Steps . . . 35

Part I — Start Here

Chapter 1. What is YLang?

YLang is a small compiled programming language. You write source code in text files ending with `.yl`, then use the YLang compiler to check your program, generate C, and invoke GCC or Clang to produce a native executable. It is not an interpreter: your `.yl` source is compiled before the resulting program runs.

The normal workflow has three stages:

1. **Check the source.** YLang parses the code and checks names, initialization, and types.
2. **Generate C.** The compiler translates the checked program into C that can be inspected.
3. **Build the executable.** GCC or Clang compiles that C code into a native program.

The compiler itself is implemented in C17. That implementation detail does not mean you write C when using YLang. Your source remains YLang, and the C backend is the default translation path.

What is stable in 2.0.0?

The stable core includes explicit declarations, constants, scalar types, functions, control flow, fixed-size homogeneous arrays, strings, formatted strings, standard input, text-file operations, path utilities, math helpers, and the documented command-line tools. Explicit typing means the compiler does not silently change an integer into a float to make an expression compile.

YLang diagnostics include a code, file location, line and column, a source excerpt, and a hint when available. These details are designed to make errors easier to investigate.

What is experimental?

Several features developed during the 2.0 cycle are available to evaluate but have separate compatibility guarantees: named imports, structs, enums, the optional SDL2 GUI and Game2D APIs, the NASM backend, native Windows support, and the optional YSPM package. This book marks those features clearly. Do not assume experimental syntax or APIs cannot change.

The stable language is suitable for learning programming, command-line tools, calculations, and text-processing programs. The current Game2D layer is a small learning aid, not a complete game engine. YLang does not claim Rust-level memory safety or provide a security sandbox.

Chapter 2. Installing YLang

The simplest way to start is to build YLang from its source repository. This gives you the compiler, tests, examples, and documentation in one place.

2.1 Requirements

For the normal C backend, use Linux or macOS with a C17 compiler and GNU Make. Git is useful for cloning the source. Ordinary command-line programs do not require SDL2; GUI and Game2D builds additionally need SDL2 development files and `pkg-config`.

Native Windows support is experimental for 64-bit Windows with MinGW-w64 inside an MSYS2 MINGW64 shell. It is not full platform parity, and MSVC is not supported by this release.

2.2 Install the tools

On Fedora:

```
sudo dnf install gcc make clang git
```

For optional GUI and Game2D features:

```
sudo dnf install SDL2-devel pkgconf-pkg-config
```

On Debian or Ubuntu:

```
sudo apt install build-essential clang make git
```

For optional GUI and Game2D features:

```
sudo apt install libsdl2-dev pkg-config
```

On macOS, install the Xcode Command Line Tools and GNU Make:

```
xcode-select --install
brew install make llvm git
```

Use `gmake` instead of the system `make` for build and test commands on macOS. To build optional GUI examples, also install `sdl2` and `pkg-config` with Homebrew.

2.3 Get the source and build

Clone the official repository and enter its folder:

```
git clone https://github.com/Yassine-Jemi01/ylang.git
cd ylang
make
make test
./build/ylang --version
```

On macOS, use `gmake` in place of `make`. The compiler is written to `build/ylang`. Running the test suite is recommended before learning the language because it verifies that your local toolchain is usable.

If a command fails, inspect the first error before trying random fixes. A missing compiler or dependency can produce multiple follow-up errors.

2.4 Install it (optional)

You can run the compiler from the checkout with `./build/ylang`. To install it system-wide, use the repository's install target:

```
sudo make install
```

A user-local install can avoid modifying system directories:

```
make install PREFIX="$HOME/.local"
```

Ensure `$HOME/.local/bin` is on your `PATH` if you want to invoke `ylang` from any folder. Check the current README for the exact installation target and prefix supported by your version.

2.5 Verify the version

```
ylang --version
```

The compiler should report version `2.0.0`. If you see an older version, your shell may be finding another executable earlier in `PATH`. Compare `command -v ylang` with `./build/ylang --version` to diagnose which one you are running.

Chapter 3. Your First Program

Create a file named `hello.yl`:

```
function main() -> int {
    print("Hello, YLang!");
    return 0;
}
```

Build and run it:

```
ylang check hello.yl
ylang build hello.yl -o hello
./hello
```

Expected output:

```
Hello, YLang!
```

If YLang is not installed, replace `ylang` with `./build/ylang` in the first two commands.

Read every line

`function main() -> int` declares the program entry point. The keyword `function` begins a function, `main` is its name, and `-> int` says the function returns an integer.

The brace `{` begins the function body; `}` ends it. YLang uses braces to group statements into blocks. The semicolon marks the end of each ordinary statement.

`print("Hello, YLang!");` writes the text followed by a newline. `return 0;` ends the function and returns zero, conventionally signalling successful completion to the operating system.

Why check first?

`ylang check hello.yl` parses the code and performs semantic checks without invoking the native C compiler. It is a fast way to get feedback while editing.

`ylang build hello.yl -o hello` validates the source and then creates an executable at the path provided after `-o`. Every time you change the source, rebuild before running it; otherwise, you may still be launching the old executable.

Try changing the greeting, running `check`, rebuilding, and launching the executable again. This simple loop—edit, check, build, run—is the most useful habit to develop at the beginning.

Chapter 4. The Compiler Commands

YLang keeps its command-line interface small. Each command is intended for a specific task.

Check source code

```
ylang check hello.yl
```

This parses and checks the source without building an executable. A successful check confirms that the compiler accepts the code at the checking stage; it does not guarantee that opening a file or performing another runtime operation will succeed.

Build a program

```
ylang build hello.yl -o hello
ylang build hello.yl -o hello --cc clang
```

The first command builds using the default compiler selection. The second explicitly selects Clang for compiling the generated C. GCC or Clang must be available on `PATH`.

Inspect the generated C

```
ylang emit-c hello.yl -o build/hello.c
```

The output is useful for debugging and understanding the translation pipeline. Prefer writing and maintaining your source in YLang rather than depending on undocumented implementation details of generated C.

Get help and check the version

```
ylang --help
ylang --version
```

Use these when a tutorial mentions an unfamiliar command or when you suspect that several installations exist on your system.

The limited fix command

```
ylang fix broken.yl -o fixed.yl
ylang check fixed.yl
```

The current `fix` command only performs a high-confidence repair for the misspelling `pritn(...)` to `print(...)`. It is not a general-purpose code repair engine. Review its output and run `check` before building it; fixing spelling cannot infer what an algorithm should do.

Keep the distinction clear: `check` validates source, `build` creates an executable, `emit-c` writes generated C, and `fix` applies one narrow supported correction.

Part II — Language Foundations

Chapter 5. Variables and Constants

A variable is a named value that your program can use and update. A constant is a name whose value cannot be reassigned after initialization.

The basic declaration form is `let type name;` or `let type name = expression;`. The type comes before the variable name.

```
let int score = 100;
let string player = "Ada";
let bool game_running = true;

score = 125;
print(player, score, game_running);
```

YLang declarations make the type explicit. When you initialize a variable, the compiler checks that the expression has the declared type.

Initialize before reading

A local variable can be declared before you assign to it:

```
let int result;
result = 20 + 22;
print(result);
```

The compiler rejects a read that it cannot prove is initialized on every relevant control-flow path. This catches mistakes such as printing a local variable before assigning it.

Prefer initializing in the declaration when that makes the code easier to follow.

Constants

Use `let const` to declare a constant:

```
let const int MAX_USERS = 100;
let const string APP_NAME = "YLang Demo";

print(APP_NAME, MAX_USERS);
```

A constant needs an initial value and cannot be assigned again. Constants are useful for limits, labels, and values that should not accidentally change in a long function.

Names, scope, and globals

Names are case-sensitive: `score` and `Score` are different. Duplicate declarations in the same scope are errors. A declaration inside an inner block can shadow a name from an outer block, but too much shadowing makes a program difficult to read.

Globals are declared outside functions. A global with no explicit initializer receives a type-appropriate zero or empty default. Explicit initialization is usually easier to understand:

```
let int total = 0;

function add_point() -> int {
    total += 1;
    return total;
}

function main() -> int {
    print(add_point());
    return 0;
}
```

This shares state between functions. In larger programs, consider passing values as parameters and returning results so that the data a function uses is visible in its signature.

Chapter 6. Types

A type tells the compiler what kind of value an expression represents and which operations are allowed.

Type	Meaning	Example
`int`	Signed 64-bit integer	`let int count = 12;`
`float`	64-bit floating-point value	`let float ratio = 0.5;`
`bool`	Boolean value	`let bool ready = true;`
`char`	One byte, ASCII-oriented	`let char initial = 'Y';`
`string`	Immutable string data	`let string title = "Hello";`
`void`	Function return type only	`function log() -> void { ... }`

`void` cannot be used as a variable type. It describes a function that does not return a value.

Integer and float literals

An integer such as `42` has integer type. A decimal such as `4.5` has float type. YLang also supports hexadecimal integers and scientific floating-point notation:

```
let int mask = 0xFF;
let float large = 1e3;
let float small = 2.5e-4;
```

`10` is an integer; `10.0` is a float. The difference matters when calling a math function or combining operands in an expression.

Booleans, characters, and strings

A boolean has the value `true` or `false`. Conditions require a boolean; YLang does not treat an arbitrary number as true or false.

A `char` is one byte, not a full Unicode character. YLang's string helpers are byte-oriented: the reported string length is a count of bytes, and case conversion is not Unicode-aware. A user-visible symbol in UTF-8 can occupy multiple bytes.

A `string` is immutable. String functions produce new values rather than modifying the existing value in place.

No implicit conversion

YLang does not silently convert values between types. This is invalid because the expression combines an integer and a float:

```
let int count = 5;
let float ratio = 2.0;

// Invalid: incompatible operand types.
// let float result = count + ratio;
```

Convert explicitly when that is what you intend. The next chapters introduce the conversion functions and safe string-parsing pattern.

Chapter 7. Printing and F-Strings

The `print` statement writes one or more values to standard output. It separates multiple arguments with one space and appends a newline.

```
print("Hello");
print("The answer is", 42);
```

Output:

```
Hello
The answer is 42
```

Supported printable values are `int`, `float`, `bool`, `char`, and `string`. `print` is a statement, not an expression that returns a new string.

Formatted strings

An f-string inserts values into a text template. Write the letter `f` directly before the opening quote and put an expression in braces.

```
let string name = "Mina";
let int score = 87;

print(f"Player: {name}");
print(f"Score: {score}");
```

F-strings make labels and values easier to read. Supported interpolations include variables and operators. Function calls and assignments are not supported inside interpolation expressions, so calculate complex values first:

```
let int first = 20;
let int second = 22;
let int total = first + second;

print(f"Total: {total}");
```

Use the documented escape forms `{{`, `}}`, `\\{`, and `\\}` when a literal brace must appear in an f-string instead of starting or ending an interpolation.

Debugging output

Print labels alongside values:

```
let int width = 640;
let int height = 400;
print("Width:", width);
print("Height:", height);
```

This is more informative than printing two numbers without identifying what they mean.

Chapter 8. Operators and Expressions

Operators calculate values and compare them. The type of each operand matters because YLang checks expressions explicitly.

Arithmetic

Operator	Meaning	Example
`+`	Addition	`a + b`
`-`	Subtraction	`a - b`
`*`	Multiplication	`a * b`
`/`	Division	`a / b`
`%`	Integer remainder	`a % b`

The `%` operator is for integers. Integer division does not produce a fractional result; use floating-point values or an explicit conversion when you need a decimal answer.

```
let int a = 17;
let int b = 5;

print(a + b);
print(a - b);
print(a * b);
print(a / b);
print(a % b);
```

Comparisons

Comparison operators produce a boolean:

- `==` means equal.
- `!=` means not equal.
- `<` means less than.
- `<=` means less than or equal.
- `>` means greater than.
- `>=` means greater than or equal.

Assignment is different from comparison: `score = 10;` stores a value, whereas `score == 10` tests it.

Boolean operators

Use `and` when both conditions must be true, `or` when at least one must be true, and `not` to negate a boolean.

```
let int age = 20;
let bool has_ticket = true;

if (age >= 18 and has_ticket) {
    print("Access granted.");
}
if (not has_ticket) {
    print("Buy a ticket first.");
}
```

Parentheses help explain complex logic:

```
let bool is_admin = false;\n\nif ((age >= 18 and has_ticket) or is_admin) {
    print("Continue.");
}
```

The stable core does not promise short-circuit behavior for `and` and `or`. Do not rely on the second operand being skipped to protect a side effect or invalid operation; evaluate such operations separately and check their result explicitly.

Assignment shortcuts and precedence

`=` assigns a value. `+=` adds to the current value, and postfix `++` increments a mutable numeric variable by one:

```
let int count = 0;
count += 1;
count++;
print(count);
```

These are standalone update statements, not general nested expressions.

Precedence follows conventional rules: unary operators first, multiplication/division/remainder, addition/subtraction, comparisons/equality, `and`, then `or`. When an expression is difficult to scan, use parentheses even if the compiler could infer the order.

Chapter 9. Making Decisions with if

The `if` statement chooses which block to run based on a boolean condition.

```
let int temperature = 28;

if (temperature > 30) {
    print("Hot");
} else {
    print("Not hotter than 30");
}
```

The condition appears inside parentheses. Braces mark the statements controlled by that condition.

Several possible cases

Use `else if` for more than two cases:

```
let int score = 76;

if (score >= 90) {
    print("Excellent");
} else if (score >= 70) {
    print("Good");
} else if (score >= 50) {
    print("Passed");
} else {
    print("Try again");
}
```

Conditions are checked from top to bottom. When one is true, its body executes and the rest of the chain is skipped. Put narrower or higher thresholds first, otherwise an earlier broad condition may hide a later case.

Combine conditions

```
let int age = 19;
let bool accepted_rules = true;

if (age >= 18 and accepted_rules) {
    print("Registration allowed");
}
```

The full condition must be boolean. If a compiler diagnostic says an `if` condition has the wrong type, check each operand and ensure you did not mix strings, integers, floats, and booleans accidentally.

A common mistake is using `=` where `==` was intended. Remember that assignment changes a variable, while comparison answers a yes-or-no question.

Chapter 10. Repeating Things: loop, while, and for

A loop repeats statements. YLang provides `loop()`, `while`, and `for`; choose the form that makes the stopping condition easy to see.

loop()

`loop()` repeats until the program leaves it with `break` or terminates:

```
let int count = 0;

loop() {
    print(count);
    count += 1;

    if (count >= 3) {
        break;
    }
}
```

The values printed are `0`, `1`, and `2`. If you forget a path to exit an unconditional loop, it can run forever.

while

A `while` loop repeats as long as its boolean condition remains true:

```
let int count = 1;

while (count <= 3) {
    print(count);
    count += 1;
}
```

Pay attention to where the loop variable changes. If it never changes in a way that makes the condition false, the loop will not end on its own.

for

A `for` loop groups an initializer, condition, and increment:

```
for (let int i = 0; i < 5; i = i + 1) {
    print(i);
}
```

The initializer runs once. The condition is checked before each iteration. After the body completes, the increment runs before the next check. A variable declared in the initializer is scoped to the loop.

break and continue

`break;` exits the innermost loop. `continue;` skips the rest of the current iteration and begins the next one.

```
for (let int i = 0; i < 6; i = i + 1) {
    if (i == 2) {
        continue;
    }
    if (i == 5) {
        break;
    }
    print(i);
}
```

This prints `0`, `1`, `3`, and `4`. The value two is skipped, and the loop exits before printing five.

Use `for` for familiar counting patterns, `while` when repetition depends on a condition, and `loop()` when you want an explicit exit inside the body. Before running a loop, be able to explain what changes each iteration and why the loop eventually stops.

Part III — Functions and Standard Library

Chapter 11. Functions

A function gives a name to a sequence of statements. It lets you reuse logic and divide a larger program into smaller pieces that each have a recognizable job.

```
function add(int left, int right) -> int {
    return left + right;
}

function main() -> int {
    let int answer = add(20, 22);
    print(answer);
    return 0;
}
```

The `add` function receives two integer parameters and returns an integer. Calling `add(20, 22)` supplies the arguments and produces the value returned by the function.

Return types

The declared return type must match the returned value. A non-void function must return a value on the supported control-flow paths.

```
function is_even(int value) -> bool {
    return value % 2 == 0;
}
```

A function that does not return a value can use `void`:

```
function show_message() -> void {
    print("Welcome");
}

function main() -> int {
    show_message();
    return 0;
}
```

Parameters have explicit types

Each parameter declares a type. The compiler rejects calls with a mismatched number or type of arguments. Since YLang does not perform implicit conversions, convert a value before passing it to a function that expects another type.

Function overloading is not part of the stable core. Do not define multiple functions with the same name and expect the compiler to select one based on the argument types.

Keep functions focused

Names such as `read_score`, `is_even`, and `calculate_total` tell a reader what the function is responsible for. Prefer passing values as parameters and returning results over using shared global state, unless shared state is genuinely part of the design.

Chapter 12. Arrays

An array stores multiple values of one element type under a single name. YLang's stable core provides fixed-size arrays whose length is inferred from the initializer.

```
let int scores[] = [10, 20, 30];

function main() -> int {
    print(scores[0]);
    print(scores[1]);
    print(scores[2]);
    return 0;
}
```

Array indexing starts at zero. In a three-element array, valid indices are zero, one, and two.

Read and update elements

```
let int scores[] = [10, 20, 30];

scores[1] = 42;
print(scores[0], scores[1], scores[2]);
```

The assignment changes only the selected element. Every access is bounds-checked at runtime; an invalid index terminates the program with a runtime error rather than silently reading arbitrary memory.

All elements must have the same type:

```
let string names[] = ["Mina", "Omar", "Lea"];
let bool flags[] = [true, false, true];
```

Arrays and functions

Array parameters use a declaration such as `int values[]`. The native interface passes a pointer plus a separate length value. Consult the current core specification and examples for the supported parameter forms.

Limits to remember

The stable core does not support returning arrays from functions, nested arrays, whole-array assignment, or constant arrays. An array's length cannot change at runtime. Initialize an array in its declaration and update individual elements.

If an index comes from user input or a calculation, verify that it is within the valid range before using it. A valid array declaration cannot guarantee that every future index is valid.

Chapter 13. Converting Between Types

YLang requires explicit conversions. Writing the conversion in your code makes the intended change visible to both the compiler and the reader.

Integer to float

```
let int count = 5;
let float precise_count = to_float(count);
let float half = precise_count / 2.0;

print(half);
```

Use a float literal such as `2.0` when the expression requires floating-point operands. Do not expect YLang to convert `count` automatically.

Float to integer

```
let float measurement = 12.8;
let int whole = to_int(measurement);
print(whole);
```

`to_int` truncates toward zero; it does not round to the nearest integer. Thus `12.8` becomes `12`, and `-12.8` becomes `-12`. If the float is not finite or is outside the signed 64-bit integer range, the runtime reports an error.

Values to strings

```
let int score = 42;
let string message = "Score: " + to_string(score);
print(message);
```

`to_string` supports `int`, `float`, `bool`, `char`, and `string`. For a string argument it returns that string unchanged.

Parse text carefully

Terminal input and file contents are text. Validate a numeric string before converting it:

```
let string raw = "123";

if (string.is_int(raw)) {
    let int value = string.parse_int(raw);
    print(value + 1);
} else {
    print("That is not an integer.");
}
```

`string.is_int` checks whether the entire string is a valid base-10 signed 64-bit integer. `string.parse_int` converts a valid value. Invalid or out-of-range text causes a runtime error, so check untrusted input first.

Use the same stored string for the check and the conversion, as in the example. If a value can change between validation and parsing, the second operation may fail even though the earlier check passed.

Chapter 14. The Math Library

The built-in `math` namespace provides common floating-point functions. Calls are checked by the compiler and translated into native C math-library calls by the default backend.

```
let float root = math.sqrt(81.0);
let float power = math.pow(2.0, 8.0);
let float angle = math.sin(0.0);

print(root);
print(power);
print(angle);
```

All arguments to the current math API must be floats, and the functions return floats. Write `math.sqrt(9.0)` rather than `math.sqrt(9)`. If the value you have is an integer, convert it with `to_float`.

Function groups

Unary functions include `sqrt`, `sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `sinh`, `cosh`, `tanh`, `exp`, `log`, `log10`, `floor`, `ceil`, `round`, and `abs`.

Binary functions include `pow`, `atan2`, `min`, `max`, and `hypot`. The `clamp(value, low, high)` function accepts three floats and limits the value to the provided interval. A lower bound greater than the upper bound causes a runtime error.

```
let float power = math.pow(2.0, 8.0);
let float distance = math.hypot(3.0, 4.0);
let float bounded = math.clamp(12.0, 0.0, 10.0);

print(power);
print(distance);
print(bounded);
```

The domain rules and non-finite results follow the platform C math library. Validate values before converting a math result into an integer or using it as an array index.

Chapter 15. Strings, Paths, and Input/Output

YLang provides basic string manipulation, line input, text-file I/O, and path utilities. These functions are useful for small command-line tools, but they are not a complete Unicode or binary-file framework.

Read input

```
print("What is your name?");
let string name = io.read_line();
print(f"Hello, {name}!");
```

`io.read_line()` reads a line from standard input and removes its trailing newline. At end-of-file it returns an empty string. The input is still a string, even if the user types digits.

String operations

```
let string name = "YLang";

print(string.length(name));
print(string.contains(name, "Lan"));
print(string.starts_with(name, "Y"));
print(string.ends_with(name, "g"));
print(string.upper(name));
print(string.lower(name));
```

The available helpers include:

- `string.length(value)` returns bytes, not Unicode characters.
- `string.contains(value, needle)` checks for a substring.
- `string.starts_with(value, prefix)` and `string.ends_with(value, suffix)` check the ends of the string.
- `string.concat(left, right)` returns a new concatenated string.
- `string.replace(value, needle, replacement)` replaces non-overlapping matches; an empty search string leaves the original unchanged.
- `string.upper(value)` and `string.lower(value)` return copies using byte-oriented case mapping from the C locale.

String equality is case-sensitive and ordering is bytewise lexicographic. Case conversion is not Unicode-aware. Repeated concatenation or case conversion in a long-running loop can increase memory use because generated strings are tracked until process exit.

Read and write text files

```
let string contents = io.read_file("notes.txt");
print(contents);
let bool saved = io.write_file("copy.txt", contents);

if (not saved) {
    print("Could not write the file.");
}
```

`io.read_file(path)` reads the entire text file into a string. An open/read failure or an embedded NUL byte terminates the program with a runtime error. Binary files are not supported by this string interface.

`io.write_file(path, content)` creates or overwrites a text file and returns `true` on success or `false` on failure. It does not create missing parent directories. Paths are interpreted relative to the process working directory, not automatically relative to the source file.

Path helpers

```
let string file = "assets/logo.png";

if (path.exists(file)) {
    print(path.basename(file));
    print(path.extension(file));
}
```

`path.exists(path)` tests whether the path can be inspected with `stat`. It may be true for a file, directory, or another filesystem object. `path.basename(path)` returns the final component. `path.extension(path)` returns the suffix after the final dot without the dot; a leading dot alone is not an extension. These helpers use POSIX path separators.

Open a file in a desktop application

`image.open(path)` launches the operating system's default opener. On Linux this is typically `xdg-open`; on macOS it uses `open`. The function does not decode an image or draw it inside a YLang GUI window. It needs a desktop session and a registered handler.

```
let string image_path = "picture.png";
let bool opened = image.open(image_path);

if (not opened) {
    print("Could not open the file.");
}
```

Treat filesystem failures as normal possibilities. Check a path when useful and always inspect return values from functions that report success or failure.

Part IV — Errors and Extensions

Chapter 16. Reading Compiler Errors and Using fix

A diagnostic is the compiler's explanation of why it cannot accept a program. YLang errors contain a code such as `E2001`, the source path, a line and column, a source excerpt, a caret, and a hint when available.

Read the first error carefully. Later messages may be consequences of the first mistake. Fix a single issue, run `ylang check` again, and repeat until the compiler accepts the source.

Type mismatches

Consider this function:

```
function double_value(int value) -> int {
    return value * 2;
}

function main() -> int {
    let float amount = 2.5;
    // double_value requires int, not float.
    print(double_value(to_int(amount)));
    return 0;
}
```

The conversion makes the types match. Remember that `to_int(2.5)` truncates to `2`, so the result is `4`. If rounding is what you intend, choose and implement the rounding step deliberately rather than assuming `to_int` rounds.

The fix command

```
ylang fix broken.yl -o fixed.yl
ylang check fixed.yl
```

The current fix operation is deliberately narrow: it corrects `pritn(...)` to `print(...)`. It is not a general automatic repair engine. Review the output and run `check` on it before building.

A reliable debugging routine

1. Run `ylang check` to get fresh diagnostics.
2. Find the first reported location and inspect the surrounding statement.
3. Verify spelling, declaration order, types, initialization, and argument counts.
4. Change one thing and check again.
5. When the source checks, build and run it.
6. If the failure appears only while running, investigate runtime conditions such as division by zero, invalid array indices, or file errors.

When asking for help, include the exact version, operating system, compiler, command, a minimal source file, expected behavior, and actual behavior.

Chapter 17. Runtime Errors

A program can pass compiler checks and still fail at runtime. Some operations depend on values supplied by the user, filesystem state, or calculations that cannot always be known statically.

Arithmetic failures

Signed integer overflow, integer division or remainder by zero, and floating-point division by zero are handled by generated runtime helpers. These failures terminate the program with an error message and exit status `70`.

If a divisor comes from input, check it before dividing. A successful type check does not tell you that a value is non-zero.

Array bounds failures

A three-element array has valid indices `0`, `1`, and `2`. Accessing index `3` is invalid. YLang performs a runtime bounds check and terminates with a runtime error instead of silently reading arbitrary memory.

Validate indices derived from calculations or external input before accessing the array.

Conversion and input failures

`to_int` can fail if a float is non-finite or outside the signed 64-bit integer range. `string.parse_int` fails for invalid or out-of-range text. Validate untrusted text with `string.is_int` before parsing.

File failures

`io.read_file` terminates the program if it cannot open/read the file or if the contents include an embedded NUL byte. `io.write_file` returns a boolean that indicates whether the write succeeded. Check the return type of the particular function you use instead of assuming all file operations fail in the same way.

YLang does not currently provide structured exception handling or implemented `Result<T, E>` values. The repository's Option and Result document is a design proposal, not an implemented core feature.

What runtime checks do not mean

These guards cover specific documented operations. They do not make YLang memory-safe at Rust's level, and they do not provide a security sandbox. Successful compilation must not be treated as proof that a program can safely process hostile data.

Chapter 18. A Complete Program

This example combines input, integer validation, a loop, a helper function, arithmetic, an explicit conversion, and formatted output. It asks for three integer scores and displays their sum and average.

```
function read_score(int position) -> int {
    loop() {
        print(f"Enter score {position}:");
        let string raw = io.read_line();

        if (string.is_int(raw)) {
            return string.parse_int(raw);
        }

        print("Please enter a whole integer.");
    }
}

function main() -> int {
    let int first = read_score(1);
    let int second = read_score(2);
    let int third = read_score(3);
    let int total = first + second + third;
    let float average = to_float(total) / 3.0;

    print(f"Total: {total}");
    print(f"Average: {average}");
    return 0;
}
```

Build and run:

```
ylang check scores.yl
ylang build scores.yl -o scores
./scores
```

Follow the data

`read_score` receives the number of the score to ask for. The loop reads a line of text. If the string is a valid signed 64-bit integer, it parses and returns the number. If the user enters invalid text, the function reports the problem and repeats the prompt.

`main` calls the helper three times and stores each result in an integer variable. The total is the sum of those values. To calculate a fractional average, the program converts the total to a float and divides by `3.0`. It does not rely on implicit conversion.

Improve it

Try rejecting negative values. Add a fourth score. Create a separate function that calculates a total from a fixed-size array. For each extension, decide what the function receives, what type it returns, and what data needs validation.

The general habit is to make each step explicit: validate external input, use matching types, split work into functions, and test with both valid and invalid values.

Chapter 19. Multi-File Imports (Experimental)

Named imports allow a file to use selected functions and global variables defined in another `.yl` file. This extension is experimental and is not included in the stable core's compatibility guarantee.

```
from "./math_tools.yl" import double_value;

function main() -> int {
    print(double_value(21));
    return 0;
}
```

The file `math_tools.yl` must define a top-level function named `double_value`. Import statements name a symbol, not a call, so omit the parentheses after the imported name.

Paths are resolved relative to the importing file first. On Linux and macOS, `YLANG_PATH` may hold a colon-separated list of library directories:

```
YLANG_PATH="$HOME/ylang-libs" ylang build main.yl -o main
```

The current implementation supports nested imports and reuses repeated references to the same canonical file. Circular imports, missing names, duplicate imports, and imports of `main()` are rejected.

Import aliases, qualified module namespaces, package manifests, dependency solving, and a package registry are not provided. The default C backend supports module graphs, but the NASM backend accepts imports only when every imported feature is in its supported subset. Read `docs/MODULE-SYSTEM-SPEC.md` before relying on imports in a large project.

Chapter 20. Structs and Enums (Experimental)

Structs and enums are experimental extensions. They are useful to learn about but should not be presented as stable core features.

Structs

A struct groups named fields into a custom type:

```
struct Point {
    int x;
    int y;
}

function shift(Point point, int dx) -> Point {
    return Point(point.x + dx, point.y);
}

function main() -> int {
    let Point origin = Point(2, 7);
    let Point moved = shift(origin, 3);
    print(moved.x, moved.y);
    return 0;
}
```

The current subset supports named fields with explicit types, positional construction in declaration order, field reads, passing and returning structs, and assignment between values of the same struct type. Fields of a mutable struct variable can be assigned values of the declared type, including a field reached through nested structs. For example, if `Hero` contains a `Position pos` field, you can write `hero.pos.x = 9;` directly. The compiler checks the final field's type and requires the root variable to be mutable; you do not need to copy the nested value and rebuild the outer struct just to update one of its fields.

Current limits include recursive by-value fields, methods, inheritance, generics, field defaults, and named constructor arguments. The C backend supports the documented struct subset; the NASM backend does not promise parity.

Enums and exhaustive matching

An enum defines a set of named alternatives:

```
enum Direction { North, East, South, West }

function main() -> int {
    let Direction facing = Direction.North;

    match (facing) {
        Direction.North -> { print("north"); }
        Direction.East -> { print("east"); }
        Direction.South -> { print("south"); }
        Direction.West -> { print("west"); }
    }
    return 0;
}
```

Each enum is a distinct type. Its unit variants can be stored in variables, passed to and returned from functions, and compared with variants of that same enum. Variants do not carry payload values in this milestone.

The `match` statement must include every variant exactly once. Missing or duplicated arms are errors. Store a value in a variable before matching if the expression would contain a call or assignment.

Payload variants, generic enums, matching bindings, and a guarantee about numeric tag values are not provided. See `docs/ENUMS.md` and `docs/STRUCTS.md` for the current reference and limits.

Part V — Extensions and Reference

Chapter 21. GUI and Game2D (Experimental)

YLang's optional GUI runtime is built on SDL2. The normal command-line workflow does not need SDL2, but builds that use GUI calls require SDL2 development files and `pkg-config`. Running an interactive window also requires a graphical session.

The GUI namespace

The current milestone supports one active window per process. The API includes:

- `gui.init()` to initialize the runtime.
- `gui.create_window(title, width, height)` to create a window.
- `gui.poll()` to process window events.
- `gui.clear(red, green, blue)` to clear the frame.
- `gui.draw_rect(x, y, width, height, red, green, blue)` to draw a filled rectangle.
- `gui.present()` to display a frame.
- `gui.last_error()` to read the latest runtime error.
- `gui.close()` and `gui.shutdown()` to release resources.

A basic loop should check failure results and release resources before leaving the program:

```
function main() -> int {
    if (not gui.init()) {
        print(gui.last_error());
        return 1;
    }

    if (not gui.create_window("YLang GUI", 800, 500)) {
        print(gui.last_error());
        gui.shutdown();
        return 1;
    }

    loop() {
        if (not gui.poll()) { break; }
        if (not gui.clear(24, 28, 38)) { break; }
        if (not gui.draw_rect(80, 80, 240, 120, 22, 160, 180)) { break; }
        if (not gui.present()) { break; }
    }

    gui.close();
    gui.shutdown();
    return 0;
}
```

RGB channels must lie between zero and 255, and rectangle dimensions must be positive. A `false` return may mean a runtime failure or a request to stop, depending on the operation. Check `gui.last_error()` for the latest explanatory message.

The Game2D helper

The experimental `stdlib/game2d.yl` module is a small source-level helper layer over the SDL2 runtime. It demonstrates event polling, a frame loop, keyboard and mouse state, elapsed-time measurement, rectangle drawing, collision checks, and a playable Dodge example.

This is not yet a full game engine. The current helper has no general sprite or image-decoding pipeline, text renderer, audio system, animation framework, physics engine, scene manager, or multiple active windows. The score in the example is printed to the terminal instead of drawn inside the window.

Read `docs/GAME2D.md` for the current API, required packages, example build command, and controls. Compilation in a headless environment does not prove interactive rendering works; a window needs a graphical session.

Chapter 22. The NASM Backend (Experimental)

The usual `ylang build` command uses the C backend. The experimental NASM backend targets only a limited subset of programs on Linux x86-64; it is not a replacement for the default backend.

On Fedora, install the assembler and linker utilities:

```
sudo dnf install nasm binutils
```

To emit an assembly file for inspection:

```
ylang emit-asm examples/your-program.yl -o build/your-program.asm
```

To assemble and link a supported source file:

```
ylang build-asm examples/your-program.yl -o build/your-program
./build/your-program
```

The tested subset includes scalar values such as `int`, `bool`, `char`, and `string`, local variables, user-defined functions with a limited number of scalar parameters, integer expressions, branches, loops, returns, and `print`.

Features such as global declarations, arrays, floats, f-strings, string comparisons, short-circuit boolean operations, and standard-library calls are outside the current subset. An assembly command failing on a valid YLang program does not automatically mean the program is invalid in the stable C backend.

Use NASM when you are specifically evaluating the experimental backend. Use the C backend for ordinary programs.

Chapter 23. Platform Support and YSPM Packages

Linux and macOS

Linux and macOS are the primary tested targets for the default stable C-backend workflow. Build the compiler with GCC or Clang, then use `ylang build` to compile your programs. GUI/Game2D development dependencies are optional.

Windows (Experimental)

Native Windows support targets 64-bit Windows with MinGW-w64 in an MSYS2 MINGW64 shell. This port has focused smoke tests; it does not promise full feature parity. MSVC is not supported. Follow `docs/WINDOWS.md` for the current installation steps and limits.

The optional YSPM package

YSPM is a Linux system package manager. YLang provides an optional `.yspkg` package for compatible YSPM systems on Linux x86-64. This package installs the compiler and related library files; it does not manage dependencies for source-level imports.

Download `ylang-2.0.0-x86_64.yspkg` from the official release and install it on a compatible system:

```
sudo yspm install ./ylang-2.0.0-x86_64.yspkg
```

The package does not automatically install GCC/Clang, GNU Make, SDL2 development files, or `pkg-config`. The host still needs GCC or Clang to compile generated C, and GUI builds need SDL2 development dependencies. The package is not a Windows or macOS installer.

Chapter 24. What YLang 2.0.0 Does Not Have

Being clear about the limitations prevents a tutorial from promising features that do not exist.

The stable core does not include implicit numeric conversions, classes, object-oriented inheritance, methods on structs, generics, implemented `Option<T>` and `Result<T, E>` types, general exception handling, or a complete LLVM/independent native backend.

Arrays cannot be returned from functions or nested inside other arrays. Constant arrays and whole-array assignment are not supported. The `char` type is one byte, and string length and case conversion are byte-oriented rather than Unicode-aware.

The experimental extension systems have additional limits. Imports do not provide a package registry, manifest, dependency solver, alias system, or qualified module namespaces. Structs and enums lack advanced forms such as methods, inheritance, payload variants, and generic variants. GUI/Game2D is a small optional layer, not a complete engine. NASM only targets its documented Linux x86-64 subset. Windows and YSPM remain experimental or platform-specific.

YLang does not claim Rust-level memory safety and does not provide a security sandbox. Do not assume that a successful compile makes the program safe for hostile inputs.

Chapter 25. Quick Reference

Declarations and functions

```
let int count = 0;
let string name = "YLang";
let const float pi_approx = 3.14;

function add(int a, int b) -> int {
    return a + b;
}

function main() -> int {
    print(add(20, 22));
    return 0;
}
```

Conditions and loops

```
if (count >= 10) {
    print("Ten or more");
} else {
    print("Less than ten");
}

while (count < 3) {
    count += 1;
}

for (let int i = 0; i < 3; i = i + 1) {
    print(i);
}
```

Arrays and conversions

```
let int values[] = [10, 20, 30];
values[0] = 99;

let float number = to_float(values[1]);
let string text = to_string(values[2]);
```

Standard library

```
let string input = io.read_line();
let bool numeric = string.is_int(input);
let float root = math.sqrt(81.0);
let bool exists = path.exists("notes.txt");
```

Remember: input is text; call `string.is_int` before `string.parse_int` on untrusted input. The math API requires floats. File operations may fail. `path.exists` can return true for a directory or another filesystem object, not only a regular file.

CLI commands

```
ylang --help
ylang --version
ylang check hello.yl
ylang build hello.yl -o hello
ylang build hello.yl -o hello --cc clang
ylang emit-c hello.yl -o build/hello.c
ylang fix broken.yl -o fixed.yl
```

The experimental assembly commands are `ylang emit-asm` and `ylang build-asm`. Use them only when the source is within the NASM backend's current supported subset.

Chapter 26. Practice and Next Steps

Work through these exercises in order. Put each solution in its own `.yl` file. Run `ylang check`, then build and run the program.

1. **Personal greeting:** read a name using `io.read_line()` and print a greeting with an f-string.
2. **Checked integer input:** read a string, call `string.is_int`, parse only after it returns true, and print the value plus one.
3. **Even or odd:** write a function that returns a boolean using `value % 2 == 0`.
4. **Counting:** print one through ten with a `for` loop.
5. **Array total:** create a fixed-size integer array and calculate the sum of its elements.
6. **Geometry:** convert integer measurements to floats and use `math.hypot` to calculate a right triangle's hypotenuse.
7. **Text processing:** read a line and report its byte length and whether it contains a selected word.
8. **File copy:** read a text file, write it to a different path, and report a failed write.
9. **Robust score input:** extend Chapter 18 to reject negative scores as well as invalid text.
10. **Experimental exploration:** once you are comfortable with the stable core, try a struct or an import and read its current reference before using it in a larger program.

When reporting a compiler bug, include the exact version, operating system, compiler, command line, minimal source file, expected behavior, and actual result. A small reproducible example is much easier to investigate than a large application.

Learn one feature at a time. Keep examples small, read diagnostics carefully, make conversions explicit, and test both valid and invalid inputs. Use the stable core for compatibility-sensitive code and treat the extension chapters as guides to experiments.

The goal of this book is not merely to list syntax. It is to help you understand why the code works, how to diagnose the mistakes, and how to grow a small example into a complete program.